
1 P1 8.1.6 The Hessian matrix

We have shown how backpropagation can be used to obtain the first derivatives of an error function with respect to the weights in the network.

Backpropagation can also be used to evaluate the second derivatives of the error, which are given by

$$\frac{\partial^2 E}{\partial w_{ij} \partial w_{kl}} \quad (8.36)$$

It is often convenient to consider all the weight and bias parameters as elements w_i of a single vector, denoted $\mathbf{w}$, in which case the second derivatives form the elements H_{ij} of the **Hessian matrix** $\mathbf{H}$:

$$H_{ij} = \frac{\partial^2 E}{\partial w_i \partial w_j} \quad (8.37)$$

where $i, j \in \{1, \dots, W\}$ and W is the total number of weights and biases.

The Hessian matrix arises in several nonlinear optimization algorithms used for training neural networks based on considerations of the second-order properties of the error surface (Bishop, 2006).

It also plays a role in some Bayesian treatments of neural networks (MacKay, 1992; Bishop, 2006) and has been used to reduce the precision of the weights in large language models to lessen their memory footprint (Shen et al., 2019).

我们已经展示了如何利用反向传播算法来计算误差函数关于网络权重的一阶导数。

反向传播同样可用于计算误差的二阶导数，其形式为：

$$\frac{\partial^2 E}{\partial w_{ij} \partial w_{kl}} \quad (8.36)$$

通常，将所有权重和偏置参数视为单一向量 $\mathbf{w}$ 中的元素 w_i 会更为方便。在这种情况下，二阶导数构成了 **Hessian 矩阵**（海森矩阵） $\mathbf{H}$ 的元素 H_{ij} ：

$$H_{ij} = \frac{\partial^2 E}{\partial w_i \partial w_j} \quad (8.37)$$

其中 $i, j \in \{1, \dots, W\}$ ，且 W 为权重和偏置的总数。

Hessian 矩阵出现在多种用于训练神经网络的非线性优化算法中，这些算法基于对误差曲面二阶性质的考量（Bishop, 2006）。

它在神经网络的某些贝叶斯处理方法中也发挥着作用（MacKay, 1992; Bishop, 2006），并且已被用于降低大语言模型中权重的精度，以减小其内存占用（Shen et al., 2019）。

这里的“海森矩阵”本质上是梯度（一阶导数）的梯度。在训练神经网络时，一阶导数告诉我们“往哪个方向走能降低误差”，而海森矩阵（二阶导数）则能告诉我们“这个方向上的误差曲面有多陡峭/多平坦”。掌握这个信息，优化算法（比如牛顿法）就能走得更聪明、收敛得更快，这也是为什么它在高级优化和模型压缩中这么有用的原因。

问：什么是 weight precision，什么是 memory footprint？

答：这两个概念是深度学习（尤其是大语言模型）中非常核心的工程与优化术语。结合上一段文献，它们的具体含义如下：

1. **Weight Precision（权重精度）**：权重精度指的是神经网络中用于存储和计算模型参数（即权重和偏置）的数据类型的精确度。它直接决定了单个参数在计算机内存中占用的字节数。

常见的精度类型：

FP32（单精度浮点数）：传统的训练精度，每个权重占用 32 位（4 字节）。

FP16 / BF16（半精度浮点数）：每个权重占用 16 位（2 字节）。目前主流的模型训练和推理都大量采用这种精度，能在几乎不损失模型性能的前提下大幅节省内存并加速计算。

INT8（8 位整数）：每个权重仅占用 8 位（1 字节）。常用于模型量化（Quantization），主要用于推理阶段。

INT4 / INT2: 更极端的低精度，目前在大语言模型的极致压缩中非常流行。

上下文联系：你上一段文本中提到的“降低大语言模型中权重的精度（reduce the precision of the weights）”，指的就是将模型从高精度（如 FP32）转换为低精度（如 INT8 或 INT4）的过程。

2. Memory Footprint（内存占用 / 内存足迹）：内存占用指的是一个程序或模型在运行（或存储）时，在计算机硬件（如 GPU 显存或 CPU 内存）中所消耗的实际空间大小。

影响因素：对于神经网络而言，内存占用主要由以下几个部分组成：

模型参数（Weights & Biases）：即模型本身的大小。

激活值（Activations）：前向传播过程中产生的中间结果。

优化器状态（Optimizer States）：如 Adam 优化器需要额外存储一阶和二阶动量。

梯度（Gradients）：反向传播时计算的导数。

上下文联系：大语言模型（LLMs）通常拥有数百亿甚至上千亿个参数，其内存占用极其庞大，往往超出单张显卡的承载能力。因此，文献中提到“lessen their memory footprint（减小其内存占用）”，正是通过降低权重精度等手段，让庞大的模型能够被塞进有限的硬件资源中。

两者的关系：简单来说，降低权重精度（Weight Precision）是手段，减小内存占用（Memory Footprint）是目的。例如，将一个包含 100 亿参数的模型从 FP32 压缩到 INT8，其参数部分的内存占用可以直接缩减为原来的四分之一。

2 P2

An important consideration for many applications of the Hessian is its efficiency with which it can be evaluated.

If there are W parameters (weights and biases) in the network, then the Hessian matrix has dimensions $W \times W$ and so the computational effort needed to evaluate the Hessian will scale like $\mathcal{O}(W^2)$ for each point in the data set.

Extension of the backpropagation procedure (Bishop, 1992) allows the Hessian matrix to be evaluated efficiently with a scaling that is indeed $\mathcal{O}(W^2)$.

Sometimes, we do not need the Hessian matrix explicitly but only the product $\mathbf{v}^T \mathbf{H}$ of the Hessian with some vector $\mathbf{v}$, and this product can be calculated efficiently in $\mathcal{O}(W)$ steps using an extension of

backpropagation (Møller, 1993; Pearlmutter, 1994).

对于 Hessian 矩阵的许多应用而言，一个重要的考量因素是其计算效率。

如果网络中包含 W 个参数（即权重和偏置），那么 Hessian 矩阵的维度将是 $W \times W$ 。因此，对于数据集中的每一个数据点，计算 Hessian 矩阵所需的计算量将按 $\mathcal{O}(W^2)$ 的复杂度增长。

通过对反向传播过程进行扩展 (Bishop, 1992)，可以高效地计算 Hessian 矩阵，其计算复杂度确实为 $\mathcal{O}(W^2)$ 。

在某些情况下，我们并不需要显式地求出完整的 Hessian 矩阵，而只需要计算 Hessian 矩阵与某个向量 $\mathbf{v}$ 的乘积 $\mathbf{v}^T \mathbf{H}$ 。利用反向传播的扩展方法，这种乘积可以在 $\mathcal{O}(W)$ 的计算步骤内高效得出 (Møller, 1993; Pearlmutter, 1994)。

3 P3

Since neural networks may contain millions or even billions of parameters, evaluating, or even just storing, the full Hessian matrix for any model is infeasible.

Evaluating the inverse of the Hessian is even more demanding as this has $\mathcal{O}(W^3)$ computational scaling.

Consequently there is interest in finding approximate alternatives to the full Hessian.

由于神经网络可能包含数百万甚至数十亿个参数，对于任何模型而言，评估甚至仅仅是存储完整的 Hessian 矩阵都是不可行的。

计算 Hessian 矩阵的逆矩阵要求则更为苛刻，其计算复杂度高达 $\mathcal{O}(W^3)$ 。

因此，寻找完整 Hessian 矩阵的近似替代方法引起了广泛的关注。

4 P4

One approximation involves simply evaluating only the diagonal elements of the Hessian and implicitly setting the off-diagonal elements to zero.

This requires $\mathcal{O}(W)$ storage and allows the inverse to be evaluated in $\mathcal{O}(W)$ steps but still requires $\mathcal{O}(W^2)$ computation (Ricotti, Ragazzini, and Martinelli, 1988), although with further approximation this can be reduced to $\mathcal{O}(W)$ steps (Becker and LeCun, 1989; LeCun, Denker, and Solla, 1990).

In practice, however, the Hessian generally has significant off-diagonal terms, and so this approximation must be treated with care.

一种近似方法是仅计算 Hessian 矩阵的对角线元素，并隐式地将非对角线元素设为零。

这种方法需要 $\mathcal{O}(W)$ 的存储空间，并允许在 $\mathcal{O}(W)$ 步内计算其逆矩阵，但仍需要 $\mathcal{O}(W^2)$ 的计算量 (Ricotti, Ragazzini, and Martinelli, 1988)。不过，通过进一步的近似，该计算量可以降低至 $\mathcal{O}(W)$ 步 (Becker and LeCun, 1989; LeCun, Denker, and Solla, 1990)。

然而在实际应用中，Hessian 矩阵通常具有显著的非对角线项，因此在应用这种近似时必须格外谨慎。

5 P5

A more convincing approach, known as the *outer product approximation*, is obtained as follows.

Consider a regression application using a sum-of-squares error function of the form

$$E = \frac{1}{2} \sum_{n=1}^N (y_n - t_n)^2 \quad (8.38)$$

where we have considered a single output to keep the notation simple (the extension to several outputs is straightforward).

We can then write the Hessian matrix in the form

$$\mathbf{H} = \nabla \nabla E = \sum_{n=1}^N \nabla y_n (\nabla y_n)^T + \sum_{n=1}^N (y_n - t_n) \nabla \nabla y_n \quad (8.39)$$

where ∇ denotes the gradient with respect to $\mathbf{w}$.

If the network has been trained on the data set and its outputs y_n are very close to the target values t_n , then the final term in (8.39) will be small and can be neglected.

More generally, however, it may be appropriate to neglect this term based on the following argument.

From Section 4.2 that the optimal function that minimizes a sum-of-squares loss is the conditional average of the target data.

The quantity $(y_n - t_n)$ is then a random variable with zero mean.

If we assume that its value is uncorrelated with the value of the second derivative term on the right-hand side of (8.39), then the whole term will average to zero in the summation over n .

翻译:

一种更具说服力的方法被称为外积近似 (outer product approximation), 其推导过程如下。

考虑一个使用平方和误差函数的回归问题, 其形式为:

$$E = \frac{1}{2} \sum_{n=1}^N (y_n - t_n)^2 \quad (8.38)$$

为简化符号, 此处我们仅考虑单个输出 (将其扩展至多个输出的情况非常直接)。

随后, 我们可以将 Hessian 矩阵写为如下形式:

$$\mathbf{H} = \nabla \nabla E = \sum_{n=1}^N \nabla y_n (\nabla y_n)^T + \sum_{n=1}^N (y_n - t_n) \nabla \nabla y_n \quad (8.39)$$

其中 ∇ 表示关于 $\mathbf{w}$ 的梯度。

如果网络已在该数据集上完成训练, 且其输出 y_n 与目标值 t_n 非常接近, 那么公式 (8.39) 中的最后一项将非常小, 从而可以被忽略。

然而, 更一般地, 基于以下论点, 忽略该项也可能是合理的。

正如第 4.2 节所述, 使平方和损失最小化的最优函数, 是目标数据的条件期望 (条件平均值)。

因此, 量 $(y_n - t_n)$ 是一个均值为零的随机变量。

如果我们假设该变量的值与公式 (8.39) 右侧的二阶导数项的值不相关，那么在对 n 求和时，整个项的均值将为零。

重点与难点解读：

1. 核心难点：Hessian 矩阵的分解

公式 (8.39) 是理解外积近似的核心。它通过求导的链式法则，将复杂的 Hessian 矩阵拆分成了两部分：第一部分仅包含一阶导数（梯度）的外积；第二部分包含了残差 $(y_n - t_n)$ 与二阶导数的乘积。

2. 重点概念：为什么可以忽略第二部分？

- **直觉层面**：当模型训练得足够好时，预测值 y_n 会无限逼近真实标签 t_n ，此时残差 $(y_n - t_n)$ 趋近于 0，第二项自然可以忽略。

- **理论层面**：这是更为严谨的统计学解释。在平方和损失下，最优模型预测的是目标数据的条件均值。这意味着预测误差（残差）的期望值为 0。只要残差与二阶导数项相互独立（不相关），在遍历整个数据集求和时，正负误差就会相互抵消，使得第二项在统计意义上趋近于零。

3. 近似的价值：通过丢弃包含二阶导数 $\nabla \nabla y_n$ 的第二项，Hessian 矩阵被近似为仅由一阶导数构成的外积形式 $\sum \nabla y_n (\nabla y_n)^T$ 。由于一阶导数（梯度）在反向传播中是现成的，这种近似极大地降低了计算和存储 Hessian 矩阵的开销，是许多现代二阶优化算法（如 Gauss-Newton 方法）的理论基础。

6 P6

By neglecting the second term in (8.39), we arrive at the **Levenberg–Marquardt approximation**, also known as the *outer product approximation* because the Hessian matrix is built up from a sum of outer products of vectors, given by

$$\mathbf{H} \simeq \sum_{n=1}^N \nabla a_n \nabla a_n^T. \quad (8.40)$$

Evaluating the outer product approximation for the Hessian is straightforward as it involves only first derivatives of the error function, which can be evaluated efficiently in $\mathcal{O}(W)$ steps using standard backpropagation.

The elements of the matrix can then be found in $\mathcal{O}(W^2)$ steps by simple multiplication.

It is important to emphasize that this approximation is likely to be valid only for a network that has been trained appropriately, and that for a general network mapping, the second derivative terms on the right-hand side of (8.39) will typically not be negligible.

翻译：

通过忽略公式 (8.39) 中的第二项，我们得到了 **Levenberg–Marquardt 近似** (Levenberg–Marquardt approximation)，它也被称为外积近似 (outer product approximation)。这是因为 Hessian 矩阵是由向量的外积之和构建而成的，其表达式为：

$$\mathbf{H} \simeq \sum_{n=1}^N \nabla a_n \nabla a_n^T. \quad (8.40)$$

计算 Hessian 的外积近似非常直接，因为它仅涉及误差函数的一阶导数。利用标准的反向传播算法，这些一阶导数可以在 $\mathcal{O}(W)$ 步内高效求出。

随后，通过简单的乘法运算，即可在 $\mathcal{O}(W^2)$ 步内求得该矩阵的各个元素。

需要特别强调的是，这种近似可能仅在网络经过充分且适当的训练后才成立。对于一般的网络映射而言，公式 (8.39) 右侧的二阶导数项通常是不可忽略的。

重点与难点解读：

1. 核心难点：符号 ∇a_n 的含义

公式 (8.40) 中的 ∇a_n 表示误差函数关于网络参数 $\mathbf{w}$ 的一阶梯度（注：原文此处使用 a_n 代表网络输出或激活值，其梯度即为误差对参数的导数）。这是该近似成立的数学基础：它将原本需要计算二阶导数的 Hessian 矩阵，巧妙地转化为了仅依赖一阶梯度的外积形式。

2. 重点概念：计算复杂度的权衡

这段话清晰地展示了该近似在工程上的巨大价值。虽然显式构建完整的 Hessian 矩阵仍需要 $\mathcal{O}(W^2)$ 的乘法和存储开销，但它完全免除了计算网络中所有参数二阶导数的巨大负担。相比于直接计算精确 Hessian 的 $\mathcal{O}(W^3)$ 复杂度，这种降维打击使得在中等规模网络中使用二阶优化方法成为可能。

3. 关键前提：近似的局限性

这是应用该方法时最容易踩坑的地方。外积近似（即 Gauss-Newton 方法的核心）在数学上等价于假设残差项 $(y_n - t_n)$ 与二阶导数项不相关，或者残差本身极小。这意味着如果网络处于训练初期（参数随机初始化，预测值与真实值相差甚远），这个近似是非常糟糕的，甚至可能导致优化方向错误。因此，它

通常只在模型收敛的后期阶段，或者作为某些自适应学习率算法（如 Levenberg-Marquardt 算法本身）的局部近似来使用。

7 P7

For a cross-entropy error function for a network with logistic-sigmoid output-unit activation functions, the corresponding approximation is given by

$$\mathbf{H} \simeq \sum_{n=1}^N y_n(1 - y_n) \nabla a_n \nabla a_n^T. \quad (8.41)$$

An analogous result can be obtained for multi-class networks having softmax output-unit activation functions.

The outer product approximation can also be used to develop an efficient sequential procedure for approximating the inverse of a Hessian (Hassibi and Stork, 1993).

翻译：

对于采用 logistic-sigmoid 输出单元激活函数的网络，其交叉熵误差函数对应的近似形式为：

$$\mathbf{H} \simeq \sum_{n=1}^N y_n(1 - y_n) \nabla a_n \nabla a_n^T. \quad (8.41)$$

对于具有 softmax 输出单元激活函数的多分类网络，也可以得到类似的结果。

此外，外积近似还可用于开发一种高效的序列过程，以近似 Hessian 矩阵的逆矩阵（Hassibi and Stork, 1993）。

重点与难点解读：

1. 核心难点：公式 (8.41) 中 $y_n(1 - y_n)$ 的来源

这一项实际上是 logistic-sigmoid 激活函数导数的体现。当使用交叉熵作为损失函数时，误差对网络参数的一阶导数中会包含激活函数的导数项。在推导外积近似时，由于交叉熵与 sigmoid 函数的完美组

合，原本复杂的二阶导数项被优雅地化简为了 $y_n(1 - y_n)$ 。这使得 Hessian 矩阵的计算可以完全复用反向传播中已经计算好的一阶梯度，极大地提升了计算效率。

2. 重点概念：多分类场景的推广

对于多分类任务，网络的输出层通常使用 softmax 激活函数配合多分类交叉熵损失。虽然 softmax 的雅可比矩阵（Jacobian）比 sigmoid 复杂，但经过类似推导，依然可以得到形式相似的外积近似。这意味着该近似方法不仅适用于二分类回归，也广泛适用于现代深度学习中最常见的多分类任务。

3. 工程应用：Hessian 逆的序列近似

在实际的优化算法（如牛顿法）中，我们往往需要的是 Hessian 矩阵的逆 $\mathbf{H}^{-1}$ 而不是 $\mathbf{H}$ 本身。直接对 $\mathcal{O}(W^2)$ 规模的矩阵求逆代价极其高昂（ $\mathcal{O}(W^3)$ 复杂度）。Hassibi and Stork (1993) 提出的序列过程（Sequential procedure）利用了外积近似的特殊结构，通过类似 Sherman-Morrison 公式的矩阵求逆引理，可以在不显式构建和求逆完整 Hessian 矩阵的情况下，随着数据样本的逐个到来，逐步更新 $\mathbf{H}^{-1}$ 的近似值。这在在线学习和模型剪枝（Pruning）等场景中具有重要的实用价值。